

Workshop No. 2

Design Artifacts and System Modeling

Computer Engineering Program

School of Engineering

Members

Tomás Mozo

Santiago Bello

Bello Edwin Marin

Helberth Rodriguez

Mateo Aguirre

Universidad Nacional de Colombia — Sede Bogotá

1. Context:

Our leasing platform is a responsive web application designed to connect people looking to rent a property with landlords seeking tenants. The platform enables users to browse and filter rental listings, view landlord and tenant profiles, and read ratings and reviews to assess each party's reputation. Users can contact one another, manage rental inquiries, and share feedback based on their experiences. Landlords manage their listings and profiles, while administrators oversee accounts, disputes, reviews, and platform analytics.
This Workshop 2 document presents the design-phase artifacts that bridge the requirements defined in Workshop 1 with implementation. The design supports all previously defined functional and non-functional requirements and user stories, with emphasis on the MVP Release 1: onboarding, profile setup, property listing and discovery, inquiries, and reputation reviews.
Key design decisions prioritize clarity, maintainability, and alignment with the non-functional requirements, including performance, data protection, accessibility, and availability. The domain model represents tenant and landlord roles and the main entities, such as User, Property, Inquiry, and Review.

2. CRC Cards:

CRC Card: User (Base Class)

Responsibilities	**Collaborators**
• Handle user registration, authentication (login/logout) and credential management (email, password hash, role) • Provide common profile attributes (fullName, contact) and basic operations (updateProfile) • Support role-based access control (LANDLORD, TENANT, ADMIN) via getRole() • Serve as foundation for inheritance by Landlord and Tenant; centralizes security logic	Landlord, tenant, Message, Review, Notification,landlordProfile, tenantProfile

CRC Card: Landlord

Responsibilities	**Collaborators**
• Get documentary support of the tenant's financial situation (RF-02) • Get the trust level of a tenant (RF-03) • Offer and keep your properties on the platform (RF-04) • Filter interested parties according to their level of trust or reputation. (RF-06) • Define what conditions the tenant must meet for your property (RF-12)	User (inherits), Reputation, Platform, Properties

CRC Card: Tenant

Responsibilities	**Collaborators**
• Finding properties that fit your needs (RF-05) • Get properties that match your preferences (RF-07) • Knowing the reputation of landlords (RF-09) • Express what he is looking for and what conditions are essential for him (RF-11) • Decide which properties are interesting for him (RF-14)	User (inherits), Reputation, Platform, Properties

CRC Card: Platform

Responsibilities	Collaborators
• Check documents and moderate user (base class) behavior (RF-10) • Get properties that match your	User (inherits), Reputation, Properties, Messages, Notifications

3. UML Diagram:

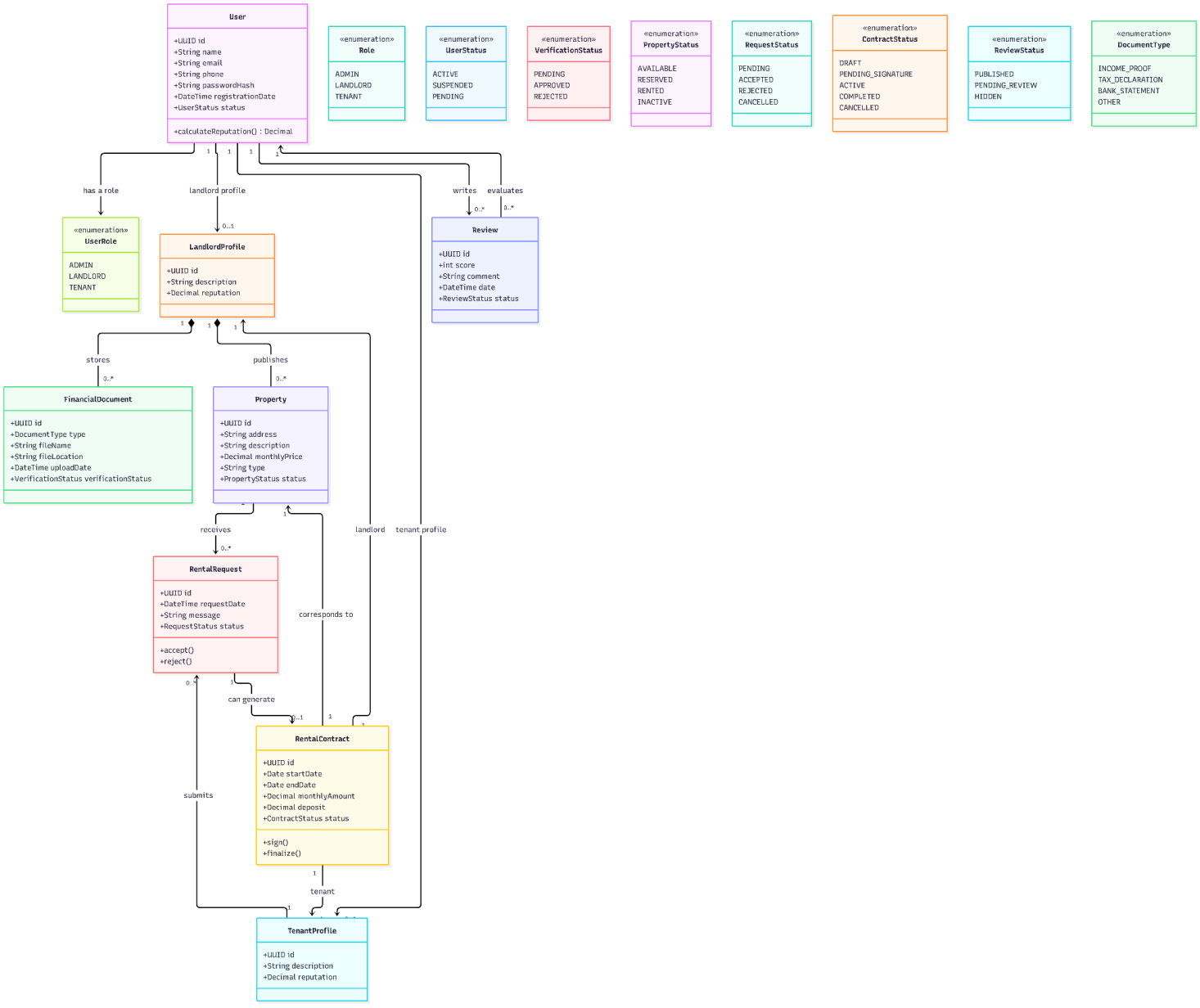


Figure 1. UML Diagram of the system.

4. Relational Database Model

4.1 Design Overview

The relational model is derived from the class diagram presented in Section 3. It targets a PostgreSQL-compatible SQL dialect (`UUID`, `TIMESTAMPTZ`, `NUMERIC`) and is normalized to Third Normal Form (3NF).

Transformation rules applied

Class-diagram element	Relational mapping
Class	One table (`snake_case`, plural name)
`UUID id` attribute	Surrogate primary key
`1` to `0..*` association	Foreign key on the "many" side
`0..1` association (profiles)	Foreign key with a `UNIQUE` constraint
Many-to-many (User ↔ Role)	Junction table with a composite primary key
Enumerations	`CHECK` constraints for status and type attributes; a lookup table (`roles`) for roles
Methods (`accept()`, `sign()`, `calculateReputation()`)	Not persisted; implemented in the application layer, and state changes update the `status` columns

Normalization notes

– **1NF:** All columns are atomic and there are no repeating groups.

– **2NF:** Every non-key attribute depends on the whole key. The only composite key (`user_roles`) has no additional attributes.

– **3NF:** There are no transitive dependencies among non-key attributes, except for two deliberate snapshot columns in `rental_contracts` (see Section 4.4).

4.2 Schema Design

Table roles

Lookup table for the `Role` / `UserRole` enumeration.

Column	Data type	Key	Constraints	Description
`role_id`	`SMALLINT`	PK	`GENERATED ALWAYS AS IDENTITY`	Role identifier
`name`	`VARCHAR(20)`		`NOT NULL`, `UNIQUE`, `CHECK (name IN ('ADMIN','LANDLORD','TENANT'))`	Role name

Table users

Column	Data type	Key	Constraints	Description
`user_id`	`UUID`	PK	`DEFAULT gen_random_uuid()`	Unique user identifier
`name`	`VARCHAR(100)`		`NOT NULL`	Full name
`email`	`VARCHAR(255)`		`NOT NULL`, `UNIQUE`, `CHECK (email LIKE '%_@_%._%')`	Login credential and contact email
`phone`	`VARCHAR(20)`		`NULL` allowed	Contact phone number
`password_hash`	`VARCHAR(255)`		`NOT NULL`	Hashed password (never plain text)

Column	Data type	Key	Constraints	Description
registration_date	TIMESTAMPTZ		NOT NULL, DEFAULT CURRENT_TIMESTAMP	Account creation date
status	VARCHAR(10)		NOT NULL, DEFAULT 'PENDING', CHECK (status IN ('ACTIVE','SUSPENDED','PENDING'))	Account state (UserStatus)

Table user_roles

Resolves the User ↔ Role relationship ("has a role"). Primary key: (user_id, role_id).

Column	Data type	Key	Constraints	Description
user_id	UUID	PK, FK → users.user_id	NOT NULL, ON DELETE CASCADE	User
role_id	SMALLINT	PK, FK → roles.role_id	NOT NULL, ON DELETE RESTRICT	Assigned role

Table landlord_profiles

Column	Data type	Key	Constraints	Description
landlord_profile_id	UUID	PK	DEFAULT gen_random_uuid()	Profile identifier
user_id	UUID	FK → users.user_id	NOT NULL, UNIQUE, ON DELETE CASCADE	Owner user (enforces the 0..1 multiplicity)
description	TEXT		NULL allowed	Landlord presentation
reputation	NUMERIC(3,2)		NOT NULL, DEFAULT 0, CHECK (reputation BETWEEN 0 AND 5)	Cached average of published review scores

Table tenant_profiles

Column	Data type	Key	Constraints	Description
tenant_profile_id	UUID	PK	DEFAULT gen random uuid()	Profile identifier
user_id	UUID	FK → users.user_id	NOT NULL, UNIQUE, ON DELETE CASCADE	Owner user (enforces the 0..1 multiplicity)
description	TEXT		NULL allowed	Tenant presentation
reputation	NUMERIC(3,2)		NOT NULL, DEFAULT 0, CHECK (reputation BETWEEN 0 AND 5)	Cached average of published review scores

Table properties

Column	Data type	Key	Constraints	Description
property_id	UUID	PK	DEFAULT gen_random_uuid()	Property identifier
landlord_profile_id	UUID	FK → landlord_profiles .landlord profile _id	NOT NULL, ON DELETE CASCADE	Publishing landlord ("publishes")
address	VARCHAR(255)		NOT NULL	Property address
description	TEXT		NULL allowed	Listing description

Column	Data type	Key	Constraints	Description
monthly_price	NUMERIC(12,2)		NOT NULL, CHECK (monthly_price > 0)	Monthly rent
type	VARCHAR(50)		NOT NULL	Property type (e.g., apartment, house, studio)
status	VARCHAR(10)		NOT NULL, DEFAULT 'AVAILABLE', CHECK (status IN ('AVAILABLE','RESERVED','RENTED','INACTIVE'))	Listing state (PropertyStatus)

Table financial_documents

Column	Data type	Key	Constraints	Description
document_id	UUID	PK	DEFAULT gen_random_uuid()	Document identifier
landlord_profile_id	UUID	FK → landlord_profiles.landlord_profile id	NOT NULL, ON DELETE CASCADE	Landlord that stores the document ("stores")
tenant_profile_id	UUID	FK → tenant_profiles.tenant_profile_id	NOT NULL, ON DELETE CASCADE	Tenant whose financial situation the document supports (RF-02)
document_type	VARCHAR(20)		NOT NULL, CHECK (document_type IN ('INCOME_PROOF','TAX_DECLARATION','BANK_STATEMENT','OTHER'))	Document category (DocumentType)
file_name	VARCHAR(255)		NOT NULL	Original file name
file_location	VARCHAR(500)		NOT NULL	Storage path or URL (file kept outside the database)
upload_date	TIMESTAMPTZ		NOT NULL, DEFAULT CURRENT TIMESTAMP	Upload date
verification_status	VARCHAR(10)		NOT NULL, DEFAULT 'PENDING', CHECK (verification_status IN ('PENDING','APPROVED','REJECTED'))	Verification state (VerificationStatus)

Table rental_requests

Column	Data type	Key	Constraints	Description
request_id	UUID	PK	DEFAULT gen random uuid()	Request identifier
property_id	UUID	FK → properties.property_id	NOT NULL, ON DELETE CASCADE	Requested property ("receives")
tenant_profile_id	UUID	FK → tenant profiles.tenant_profile_id	NOT NULL, ON DELETE CASCADE	Tenant who submits the request ("submits")
request_date	TIMESTAMPTZ		NOT NULL, DEFAULT CURRENT TIMESTAMP	Submission date
message	TEXT		NULL allowed	Message to the landlord

Column	Data type	Key	Constraints	Description
status	VARCHAR(10)		NOT NULL, DEFAULT 'PENDING', CHECK (status IN ('PENDING','ACCEPTED','REJECTED','CANCELLED'))	Request state (RequestStatus)

Additional constraint: partial unique index on (property_id, tenant_profile_id) WHERE status IN ('PENDING','ACCEPTED'), so a tenant cannot have two active requests for the same property.

Table rental_contracts

Column	Data type	Key	Constraints	Description
contract_id	UUID	PK	DEFAULT gen_random_uuid()	Contract identifier
request_id	UUID	FK → rental requests.request_id	UNIQUE, NULL allowed, ON DELETE SET NULL	Originating request ("can generate", 0..1)
property_id	UUID	FK → properties.property id	NOT NULL, ON DELETE RESTRICT	Leased property ("corresponds to")
landlord_profile_id	UUID	FK → landlord profiles.landlord_profile id	NOT NULL, ON DELETE RESTRICT	Landlord party ("landlord")
tenant_profile_id	UUID	FK → tenant_profiles.tenant_profile_id	NOT NULL, ON DELETE RESTRICT	Tenant party ("tenant")
start_date	DATE		NOT NULL	Lease start
end_date	DATE		NOT NULL, CHECK (end_date > start_date)	Lease end
monthly_amount	NUMERIC(12,2)		NOT NULL, CHECK (monthly_amount > 0)	Agreed monthly rent
deposit	NUMERIC(12,2)		NOT NULL, DEFAULT 0, CHECK (deposit >= 0)	Security deposit
status	VARCHAR(20)		NOT NULL, DEFAULT 'DRAFT', CHECK (status IN ('DRAFT','PENDING_SIGNATURE','ACTIVE','COMPLETED','CANCELLED'))	Contract state (ContractStatus)

Table reviews

Column	Data type	Key	Constraints	Description
review_id	UUID	PK	DEFAULT gen_random_uuid()	Review identifier
author_user_id	UUID	FK → users.user_id	NOT NULL, ON DELETE CASCADE	User who writes the review ("writes")
evaluated_user_id	UUID	FK → users.user_id	NOT NULL, ON DELETE CASCADE	User being evaluated ("evaluates")
score	SMALLINT		NOT NULL, CHECK (score BETWEEN 1 AND 5)	Rating
comment	TEXT		NULL allowed	Written feedback
review_date	TIMESTAMPTZ		NOT NULL, DEFAULT CURRENT_TIMESTAMP	Date of the review (date in the class diagram)

Column	Data type	Key	Constraints	Description
status	VARCHAR(15)		NOT NULL, DEFAULT 'PENDING_REVIEW', CHECK (status IN ('PUBLISHED','PENDING REVIEW','HIDDEN'))	Moderation state (ReviewStatus)

Table-level constraint: `CHECK (author_user_id <> evaluated_user_id)`, so users cannot review themselves.

Recommended indexes (performance NFR)

Table	Index	Purpose
properties	(status, monthly price)	Listing discovery and price filtering
properties	(landlord_profile_id)	Landlord's listings
rental_requests	(property_id, status), (tenant profile id)	Inquiry management
reviews	(evaluated_user_id, status)	Reputation lookup and recalculation
financial_documents	(tenant_profile_id, verification status)	Document verification by landlords and administrators

4.3 ER Diagram

The following Entity-Relationship diagram shows all tables, their keys, and the cardinality of each relationship (crow's foot notation).

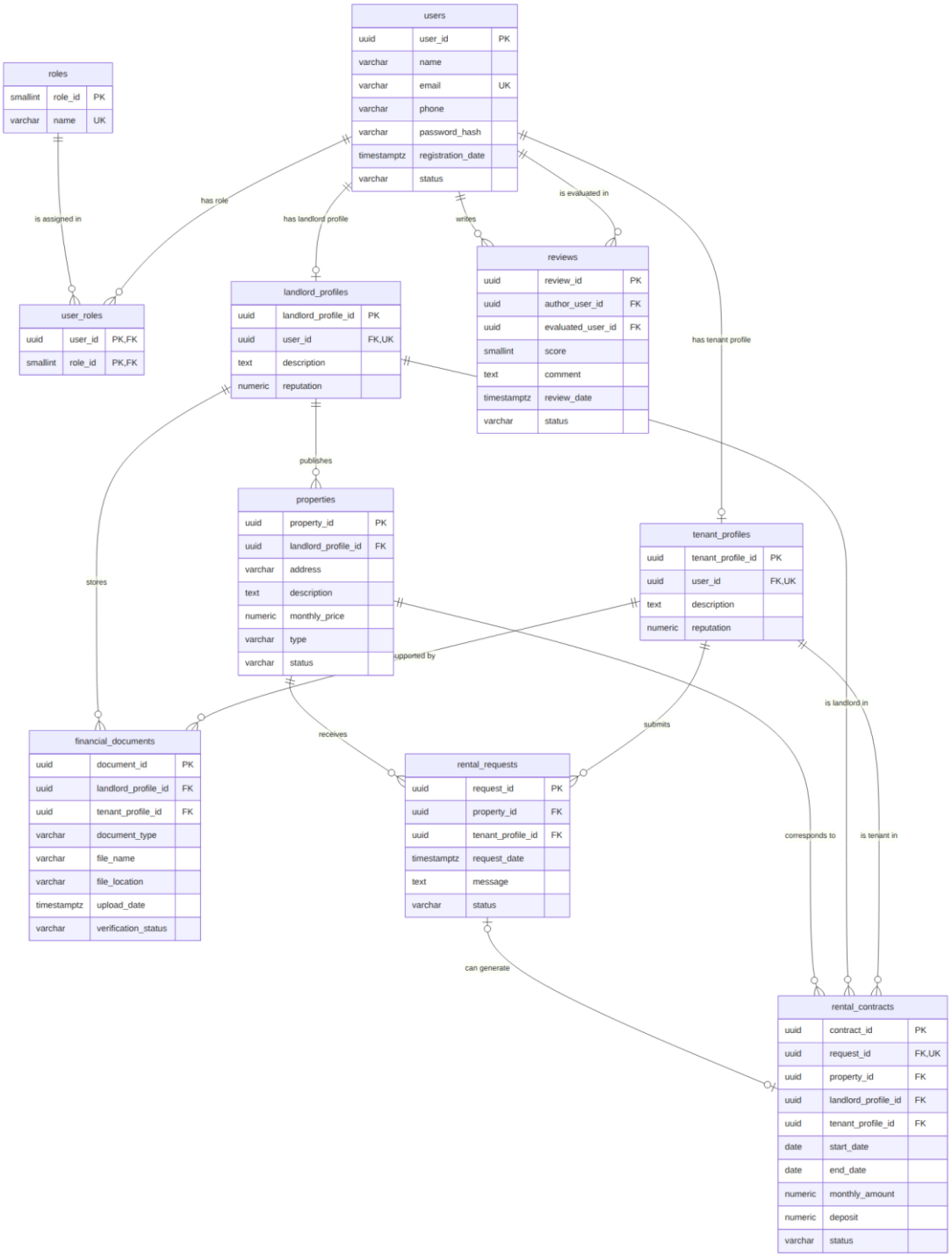


Figure 2. ER diagram of the relational model.

Relationship and cardinality summary

#	Parent (1 side)	Child (N side)	Cardinality	Type	Implemented by
1	users	user_roles	1:N	Identifying	user_roles.user_id
2	roles	user_roles	1:N	Identifying	user_roles.role_id
–	users ↔ roles	–	**N:M**	Resolved via junction table	user_roles
3	users	landlord_profiles	**1:0..1** (one-to-one)	Non-identifying	landlord_profiles.user_id (UNIQUE)
4	users	tenant_profiles	**1:0..1** (one-to-one)	Non-identifying	tenant_profiles.user_id (UNIQUE)
5	landlord_profiles	properties	1:N	Non-identifying (composition)	properties.landlord_profile_id
6	landlord_profiles	financial_documents	1:N	Non-identifying (composition)	financial_documents.landlord_profile_id
7	tenant_profiles	financial_documents	1:N	Non-identifying	financial_documents.tenant profile id
8	properties	rental_requests	1:N	Non-identifying	rental requests.property id
9	tenant_profiles	rental_requests	1:N	Non-identifying	rental_requests.tenant_profile_id
10	rental_requests	rental_contracts	**1:0..1** (one-to-one)	Non-identifying	rental_contracts.request_id (UNIQUE)
11	properties	rental_contracts	1:N	Non-identifying	rental_contracts.property_id
12	landlord_profiles	rental_contracts	1:N	Non-identifying	rental_contracts.landlord_profile id
13	tenant_profiles	rental_contracts	1:N	Non-identifying	rental_contracts.tenant_profile id
14	users (author)	reviews	1:N	Non-identifying	reviews.author_user_id
15	users (evaluated)	reviews	1:N	Non-identifying	reviews.evaluated_user_id

4.4 Business Rules and Design Decisions

1. **Role-based access:** A user can hold several roles through `user_roles`, so one account may act as both landlord and tenant. A user needs a profile row only for the roles that require one.
2. **Reputation:** `reputation` is a cached value, recomputed by `calculateReputation()` as the average `score` of `reviews` with `status = 'PUBLISHED'` for the evaluated user. It is stored on purpose to meet the performance non-functional requirement (fast filtering by reputation, RF-06 and RF-09) and is kept consistent by the application layer or a database trigger.
3. **Moderation:** New reviews start as `PENDING_REVIEW`, and only `PUBLISHED` reviews count toward reputation. Administrators can set a review to `HIDDEN`.
4. **Contract snapshots:** `rental_contracts.landlord_profile_id` and `monthly_amount` are deliberate snapshots of the landlord and the price at signing time. They are legal records that must not change if the property is later edited or reassigned.
5. **Data protection:** Passwords are stored only as hashes, and financial documents are stored as file references (`file_location`), not as binary data in the database.
6. **Deletion policy:** Profile-owned data (properties, documents, requests) is removed with `CASCADE`. Contracts use `RESTRICT` to preserve the legal history.

Software Engineering II · Workshop 2 · Section 5

Mockups

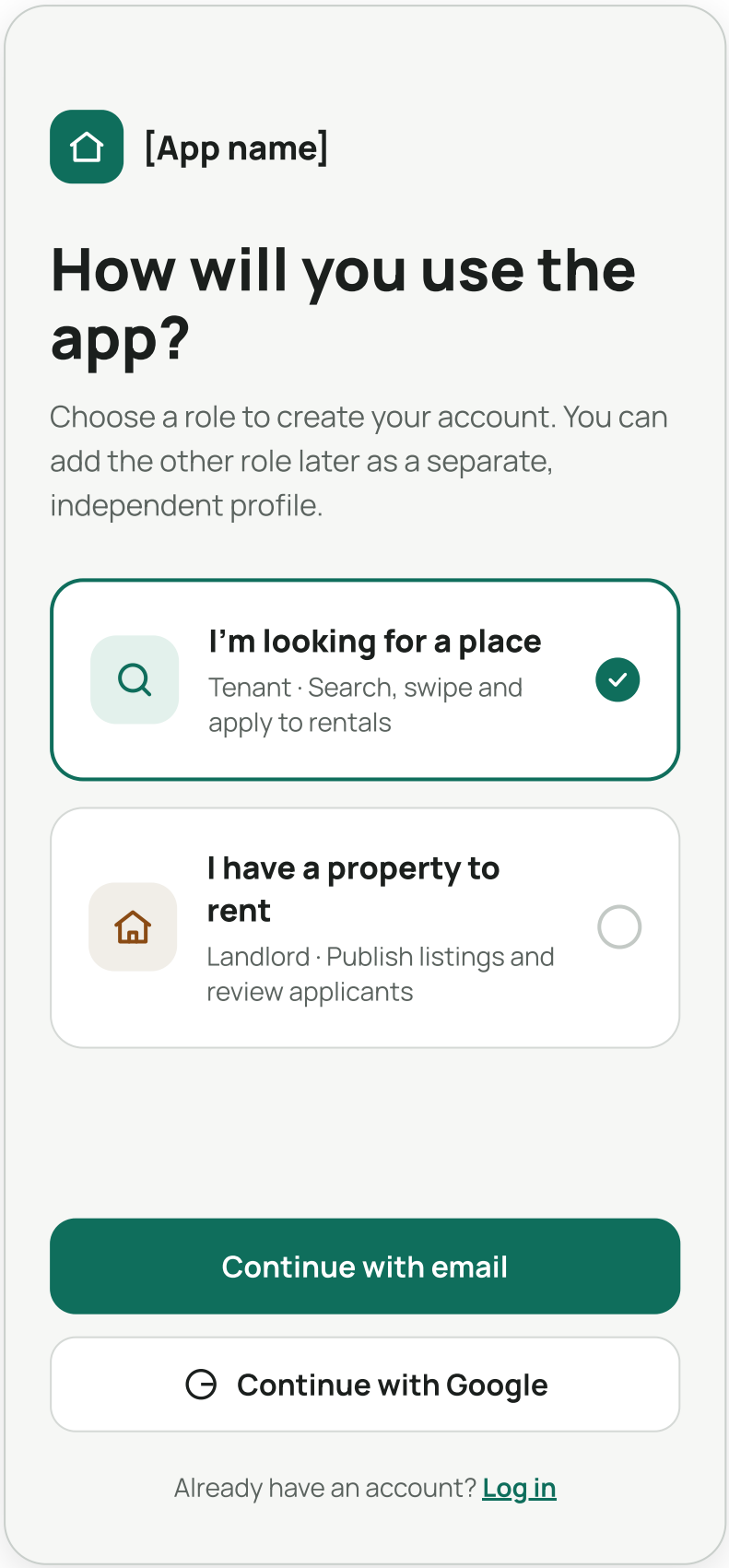
Rental Real-Estate Platform. These mid-fidelity mockups cover the main screens for the three actors: Tenant, Landlord and Administrator. Each page names the user stories (HU) and requirements (RF/RNF) the screen supports. Names, prices and scores are sample data.

01	Role selection (HU-01)
02	Profile selection (RF-01)
03	Score questionnaire (HU-02)
04	Profile score result (RF-02, RF-10)
05	Search with filters (HU-03)
06	Swipe mode (HU-04)
07	Property detail & request (HU-05)
08	Match (RF-13)
09	Chat with disclaimer (RF-07)
10	Publish property (HU-07, HU-09)
11	Property score (HU-08)
12	Official requests (HU-10)
13	Likes received (HU-11, HU-12)
14	Request verification (HU-06)
15	Post-rental rating (HU-13)
16	Interaction history (HU-14)
17	Admin panel (HU-15, HU-16, HU-17)

01 Role selection

HU-01

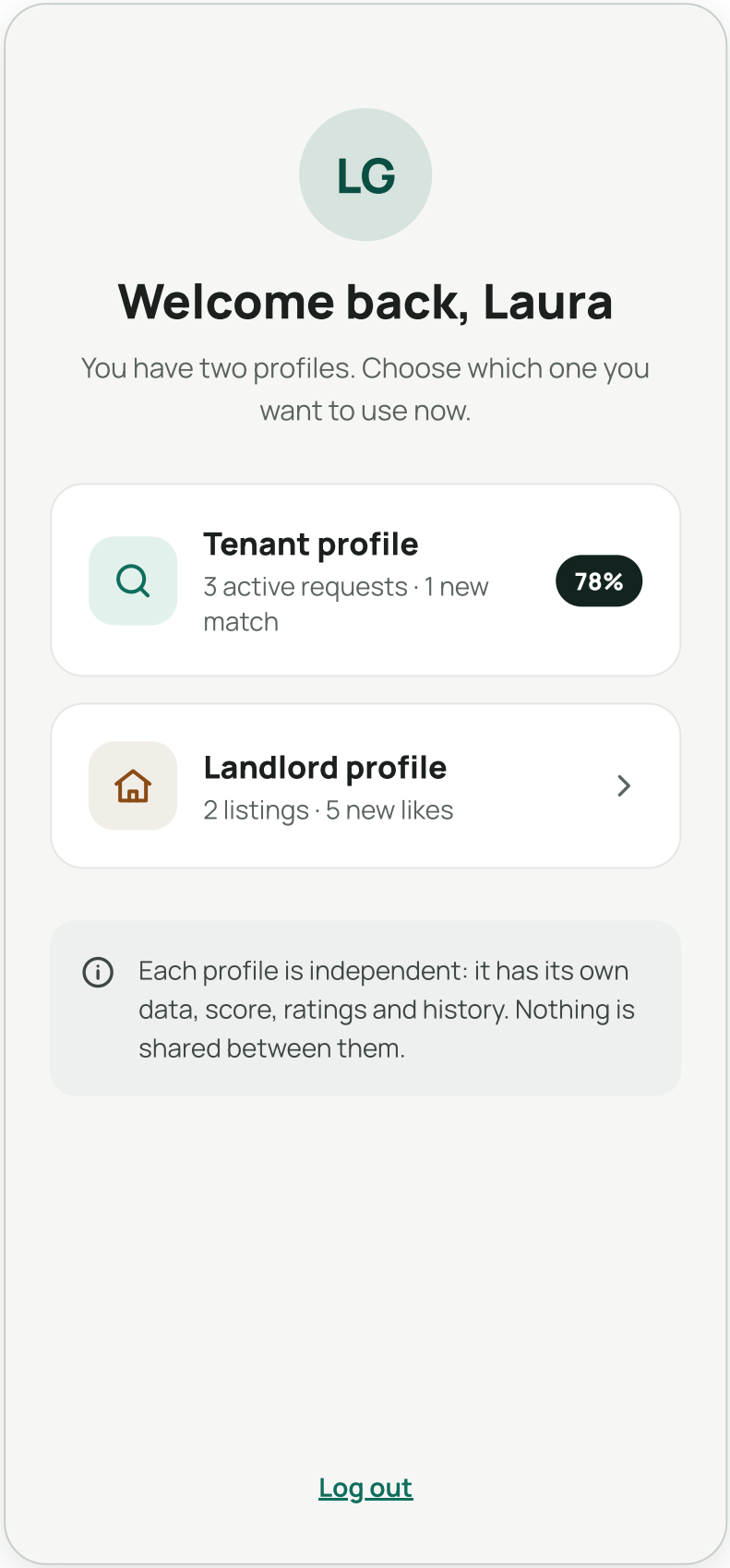
First screen for new users. They choose Tenant or Landlord before entering email or password.



02 Profile selection

RF-01

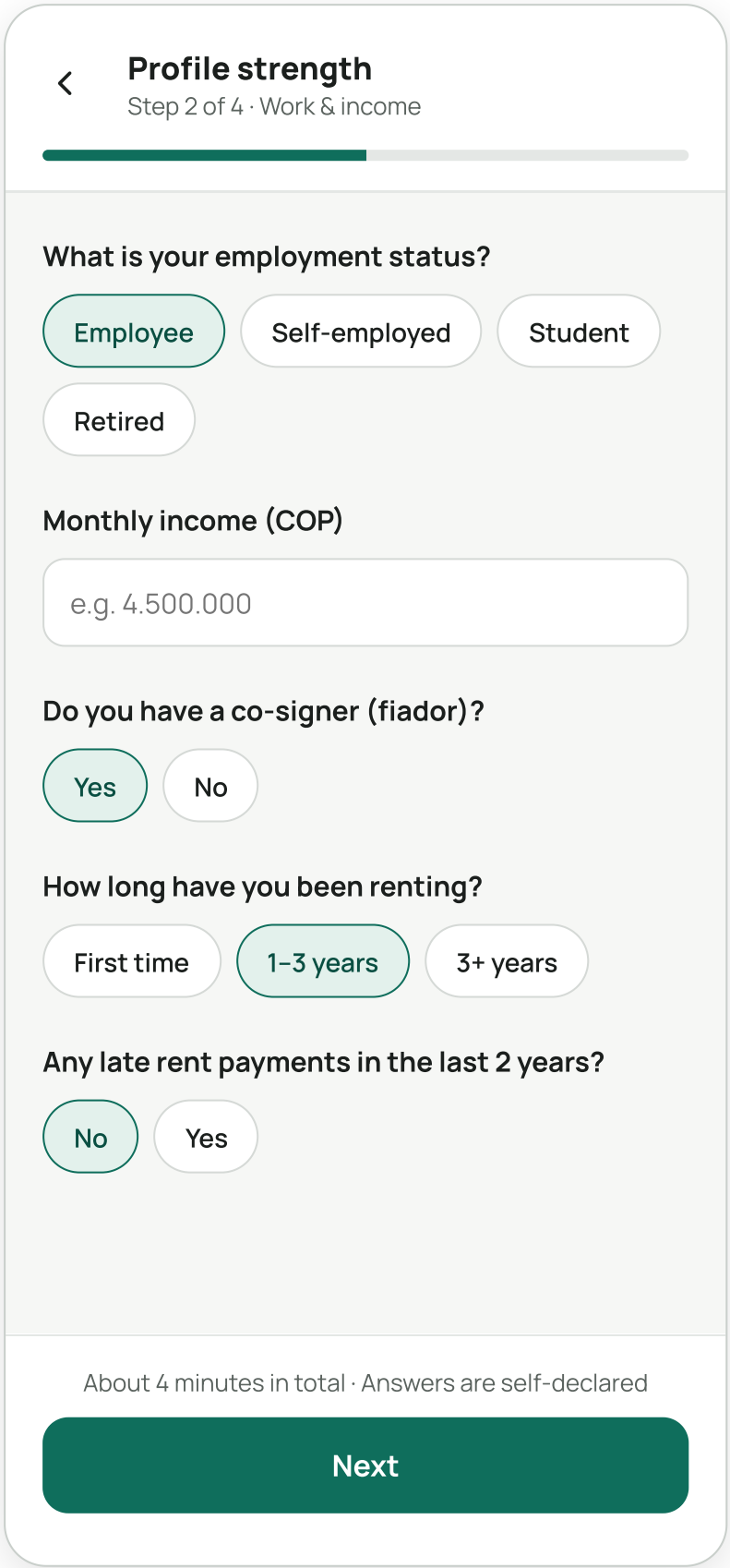
Shown when one person has both roles. They pick which independent profile to use.



03 Score questionnaire

HU-02

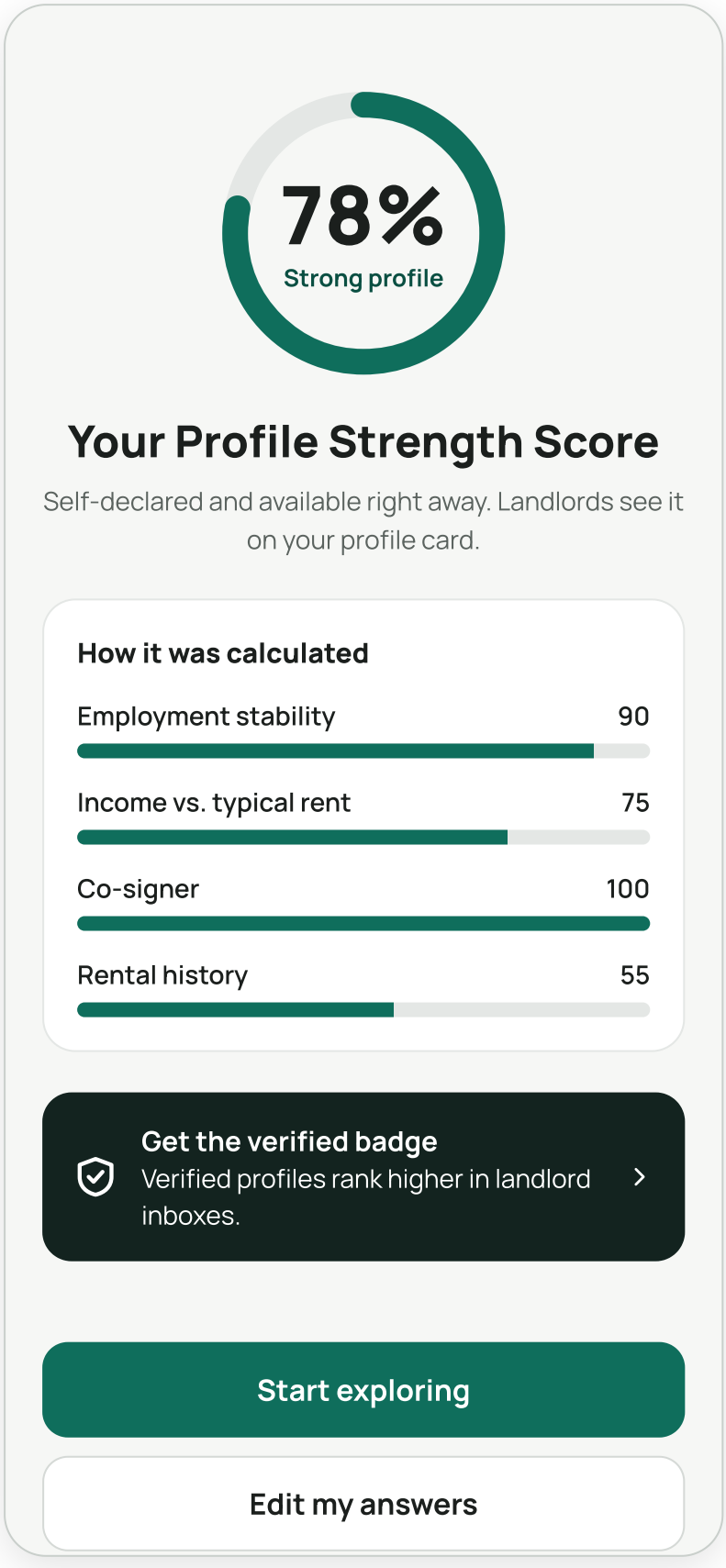
Step-by-step form about employment, income, co-signer and rental history, used to compute the tenant's score.



04 Profile score result

RF-02, RF-10

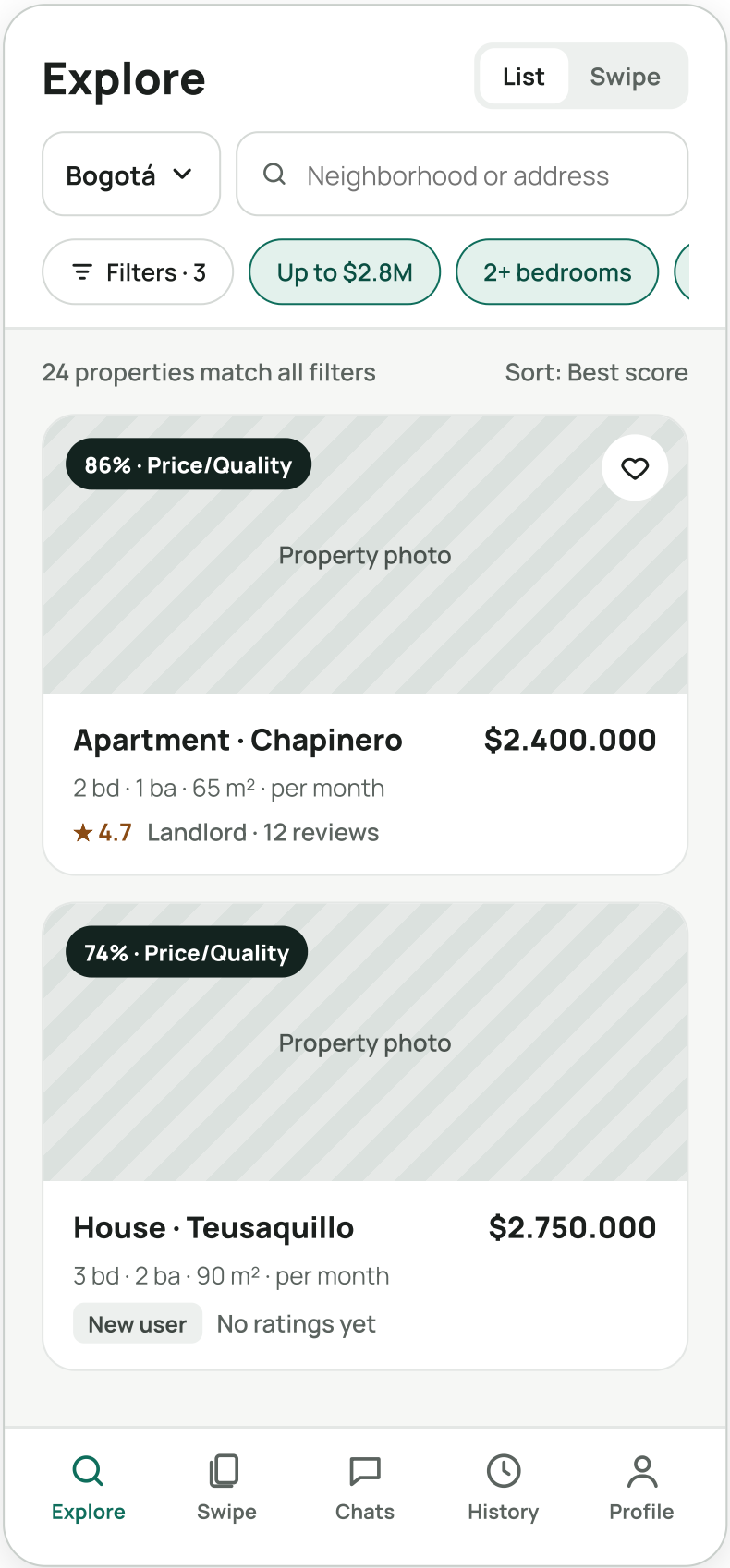
Shows the self-declared Profile Strength Score and how it was calculated, and invites the user to get verified.



05 Search with filters

HU-03

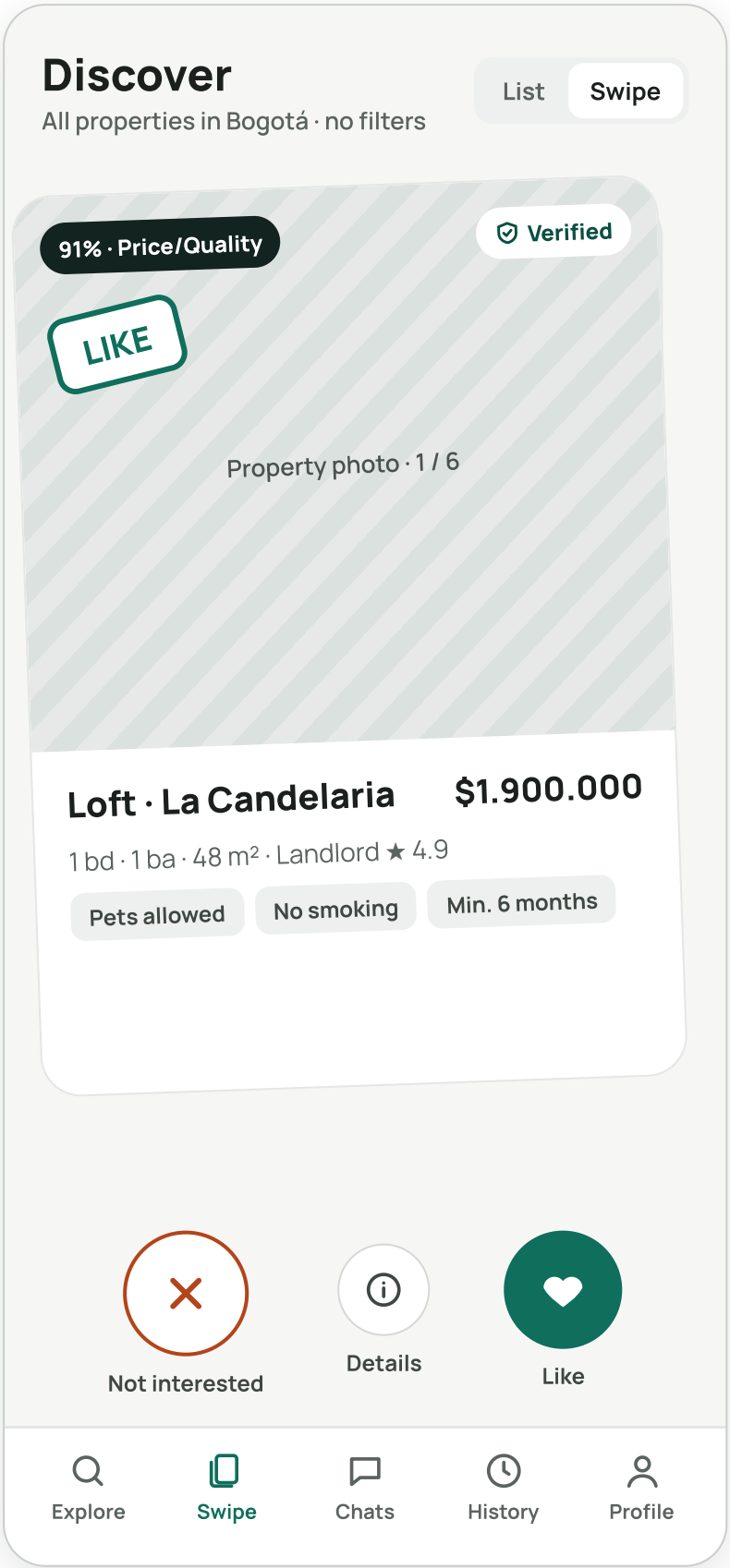
List of properties filtered by city, neighborhood, price, rooms and landlord rating. Landlords with no ratings show “New user”.



06 Swipe mode

HU-04

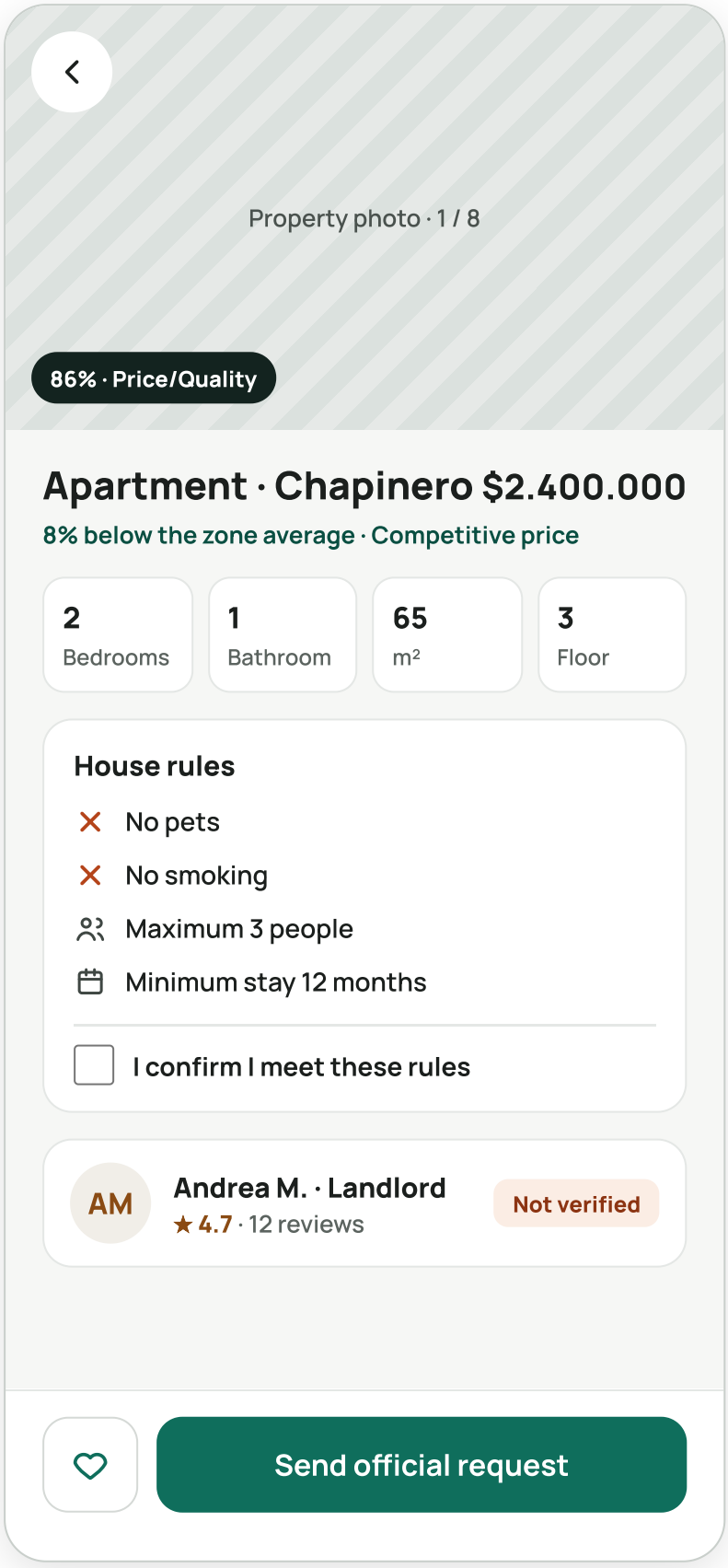
Card-by-card discovery of every property in the city. Swipe right to like, left to skip, with accessible buttons as an alternative (RNF-13).



07 Property detail & request

HU-05

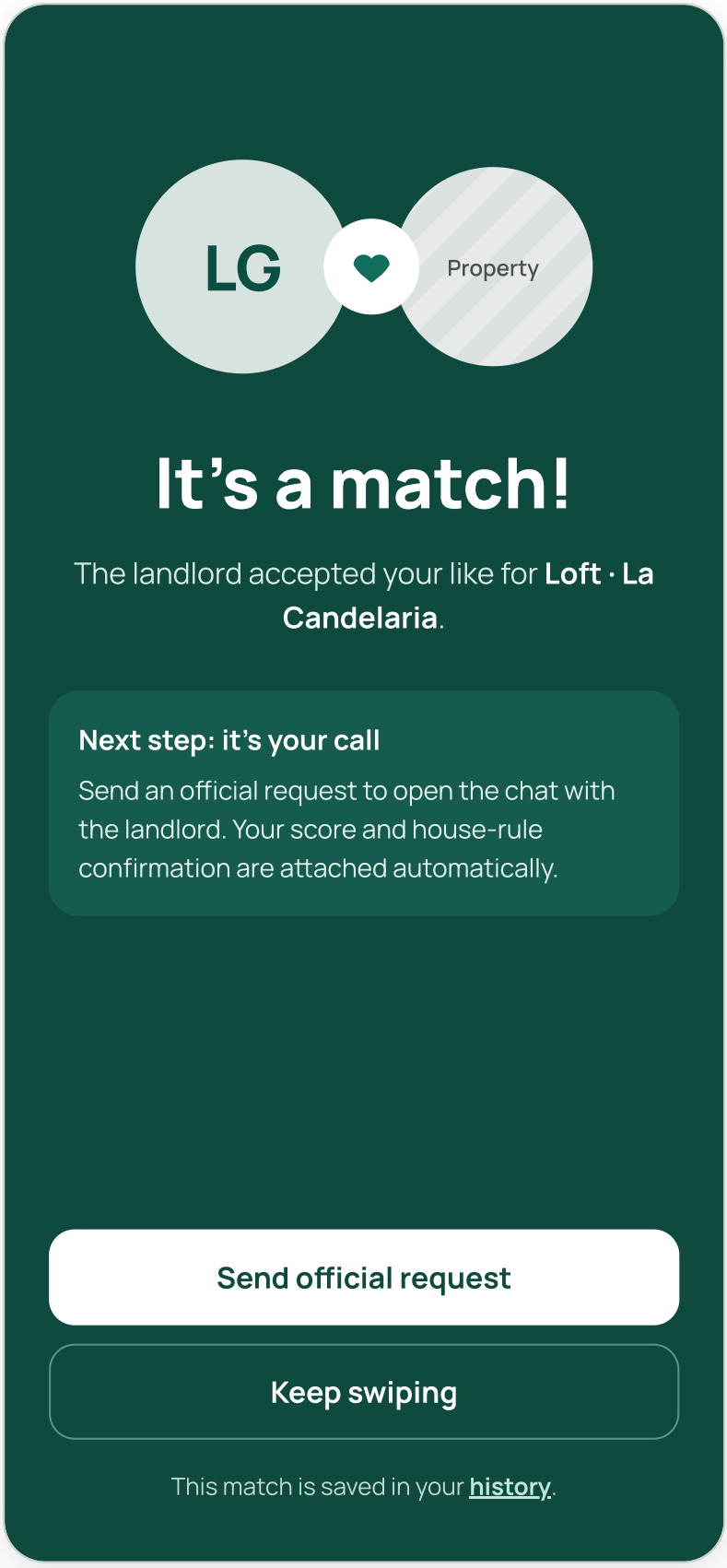
Full listing with the Price/Quality score, house rules that must be confirmed, and the “Send official request” action.



08 Match

RF-13

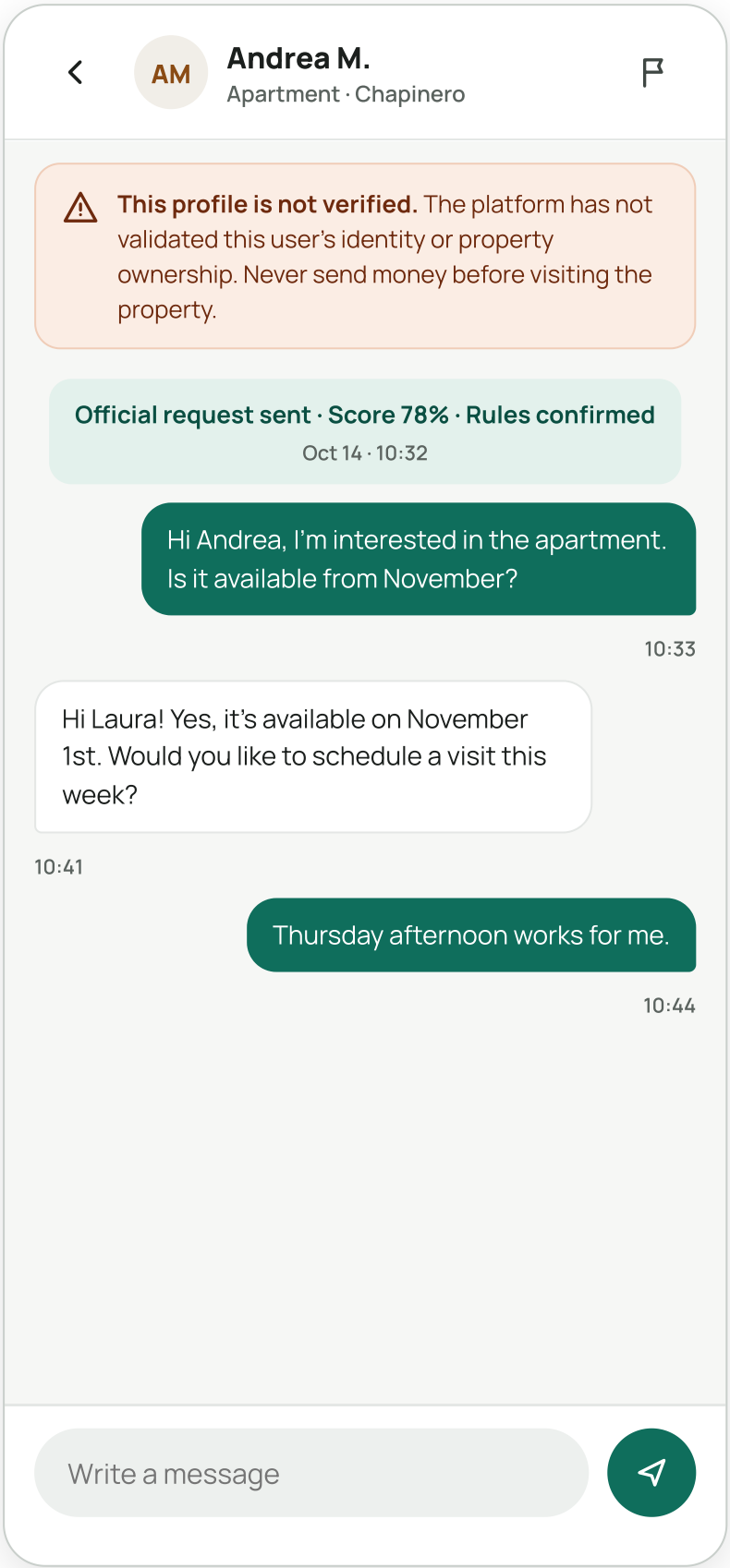
Tells the tenant that the landlord accepted their like and lets them send the official request.



09 Chat with disclaimer

RF-07

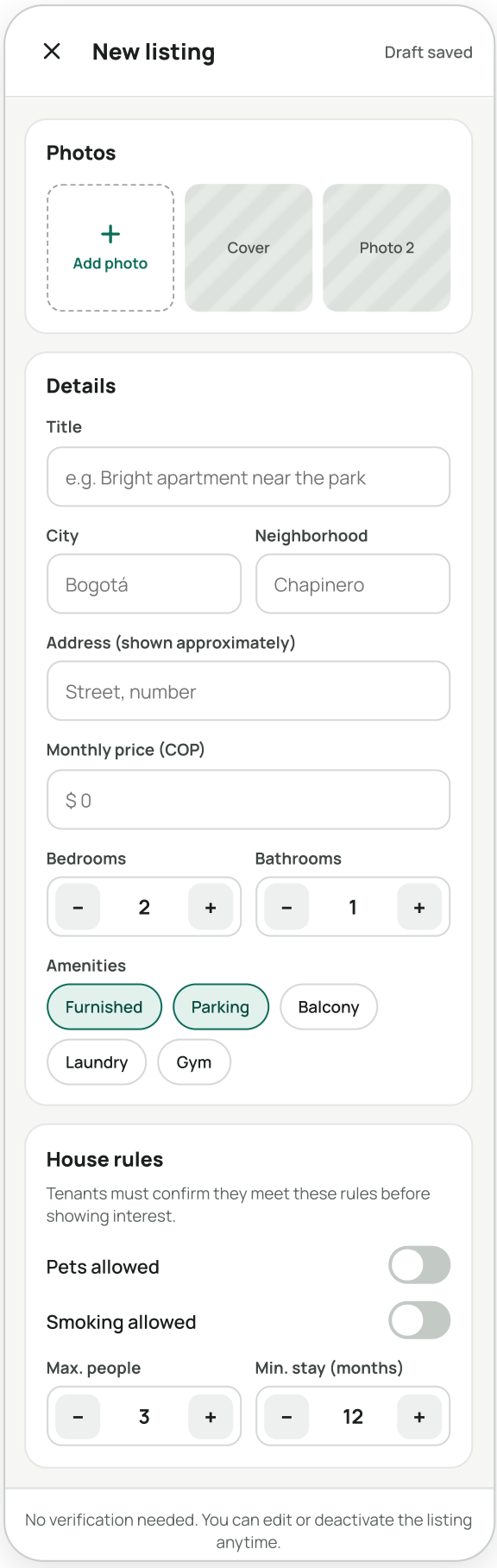
Conversation opened by an official request. Shows a legal disclaimer when the other profile is not verified.



10 Publish property

HU-07, HU-09

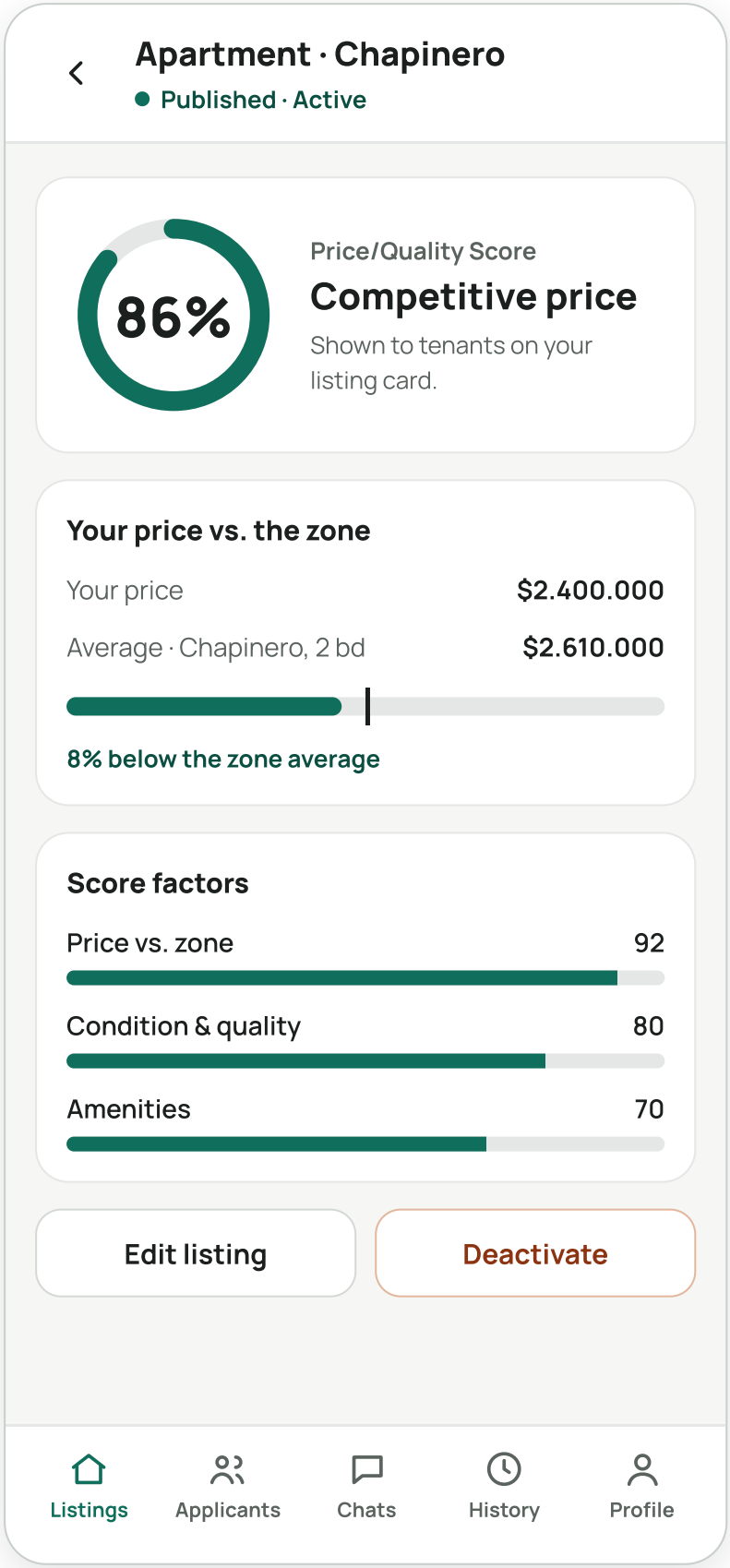
Form to publish a listing right away (photos, details, amenities, house rules), with no prior verification.



11 Property score

HU-08

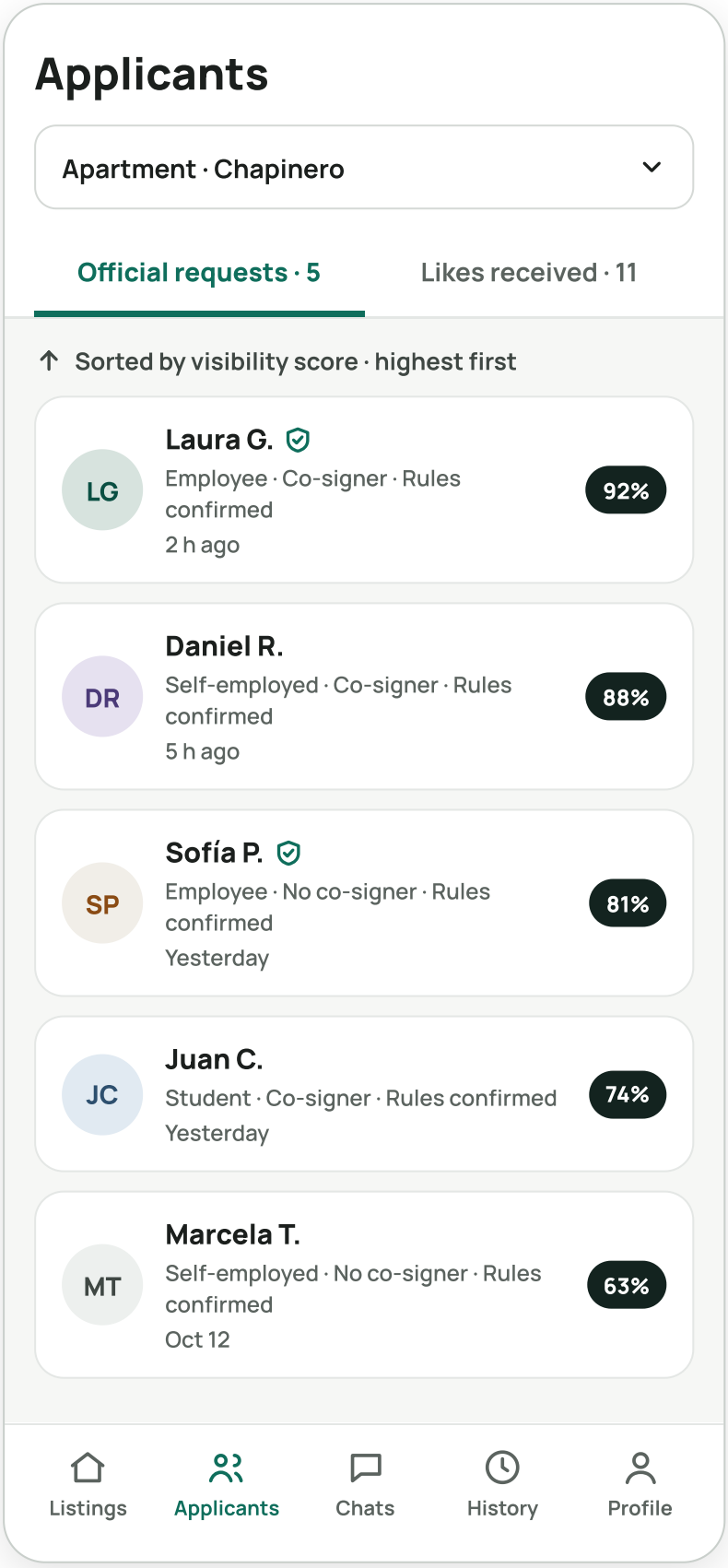
Price/Quality score compared with the zone average, plus edit and deactivate actions.



12 Official requests

HU-10

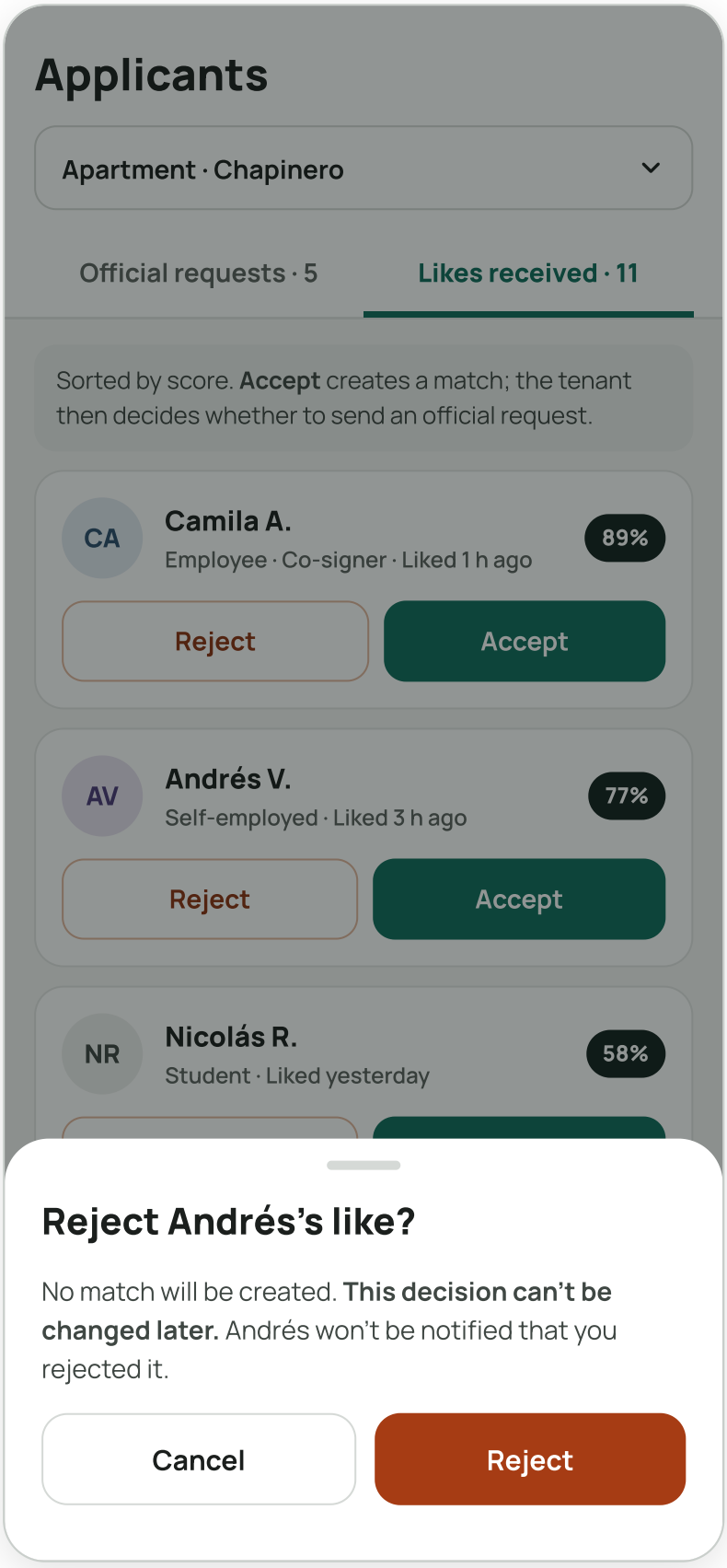
Landlord inbox of official requests, sorted by score from highest to lowest.



13 Likes received

HU-11, HU-12

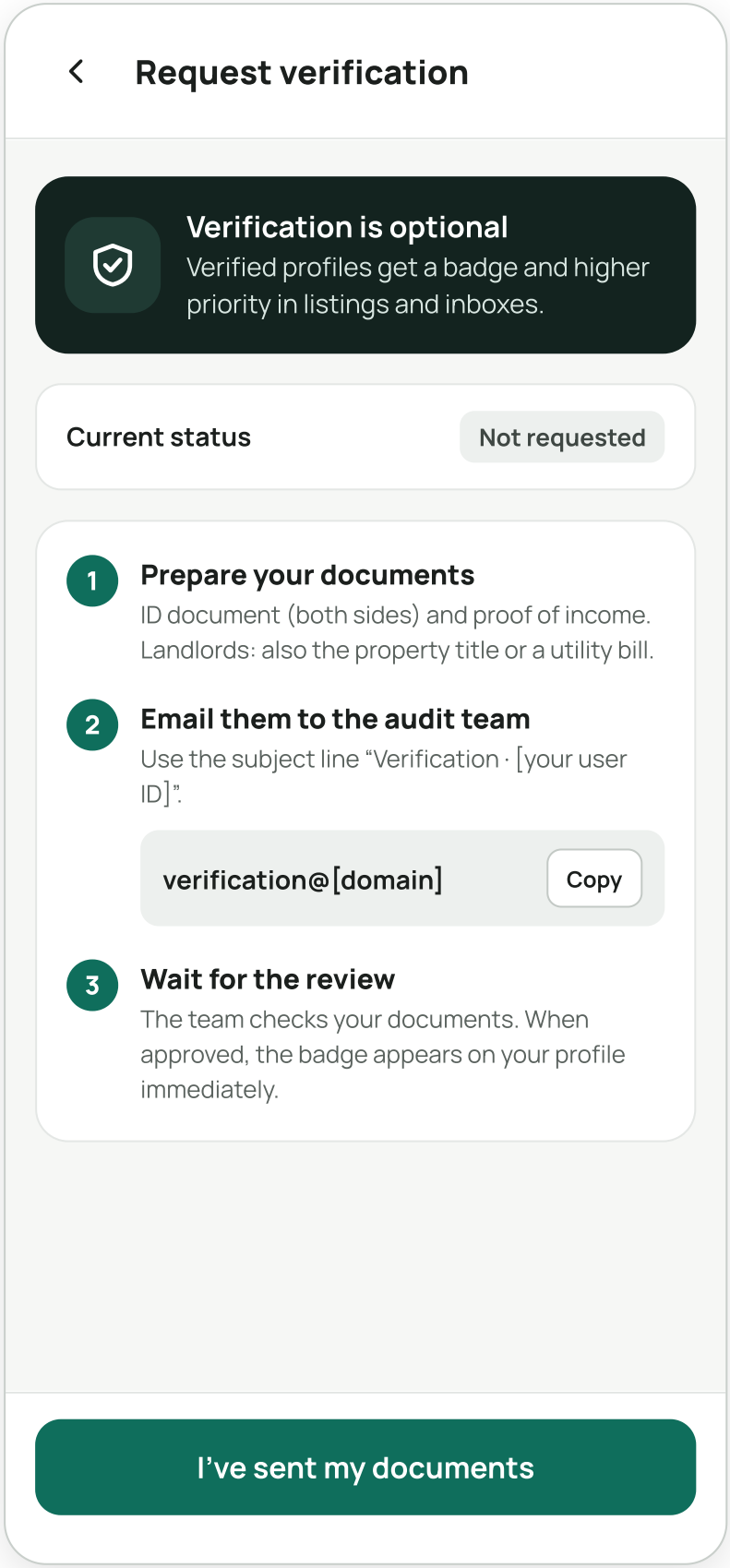
Separate inbox of likes, sorted by score, with Accept/Reject buttons. A rejection asks for confirmation and cannot be undone.



14 Request verification

HU-06

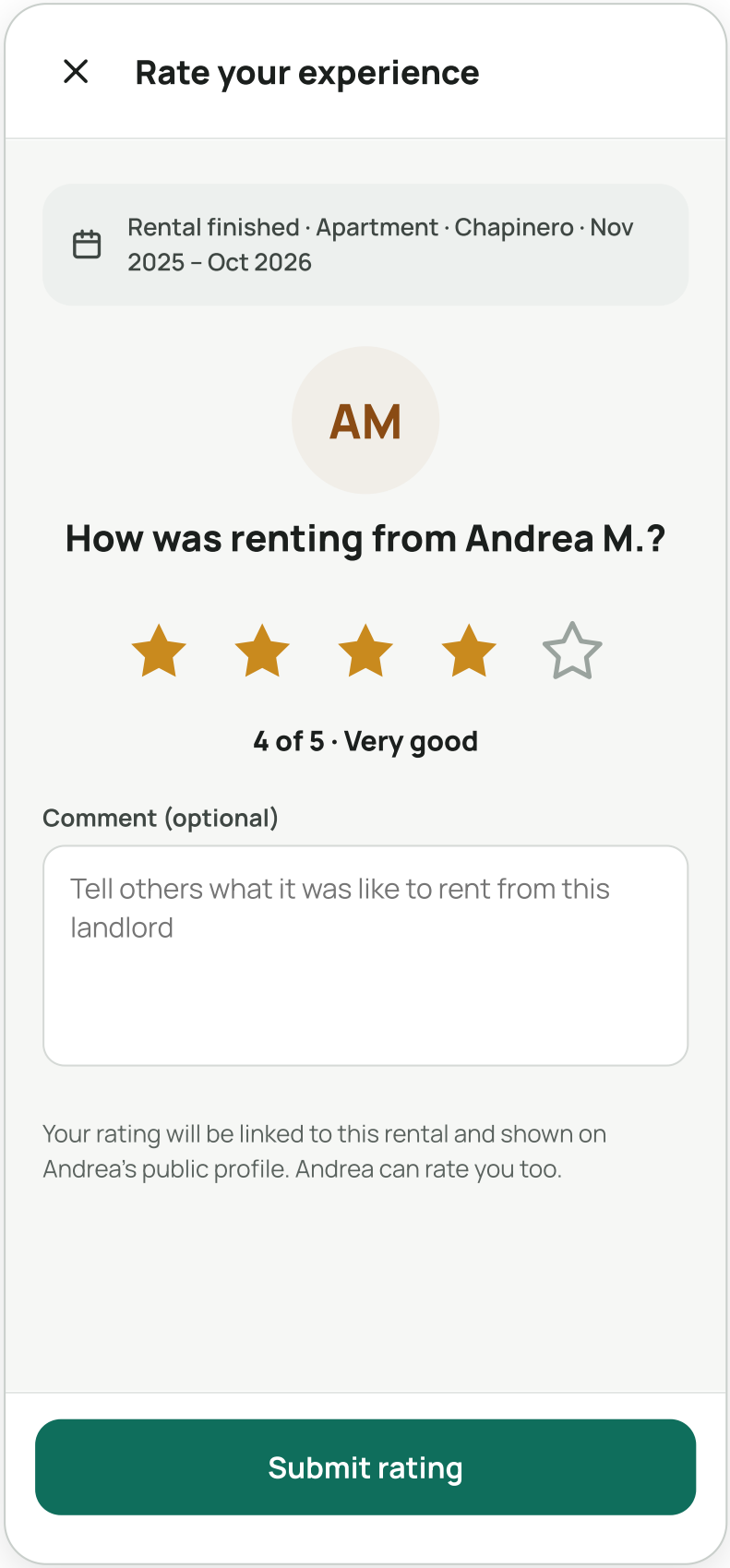
Explains the optional verification and the steps to email supporting documents.



15 Post-rental rating

HU-13

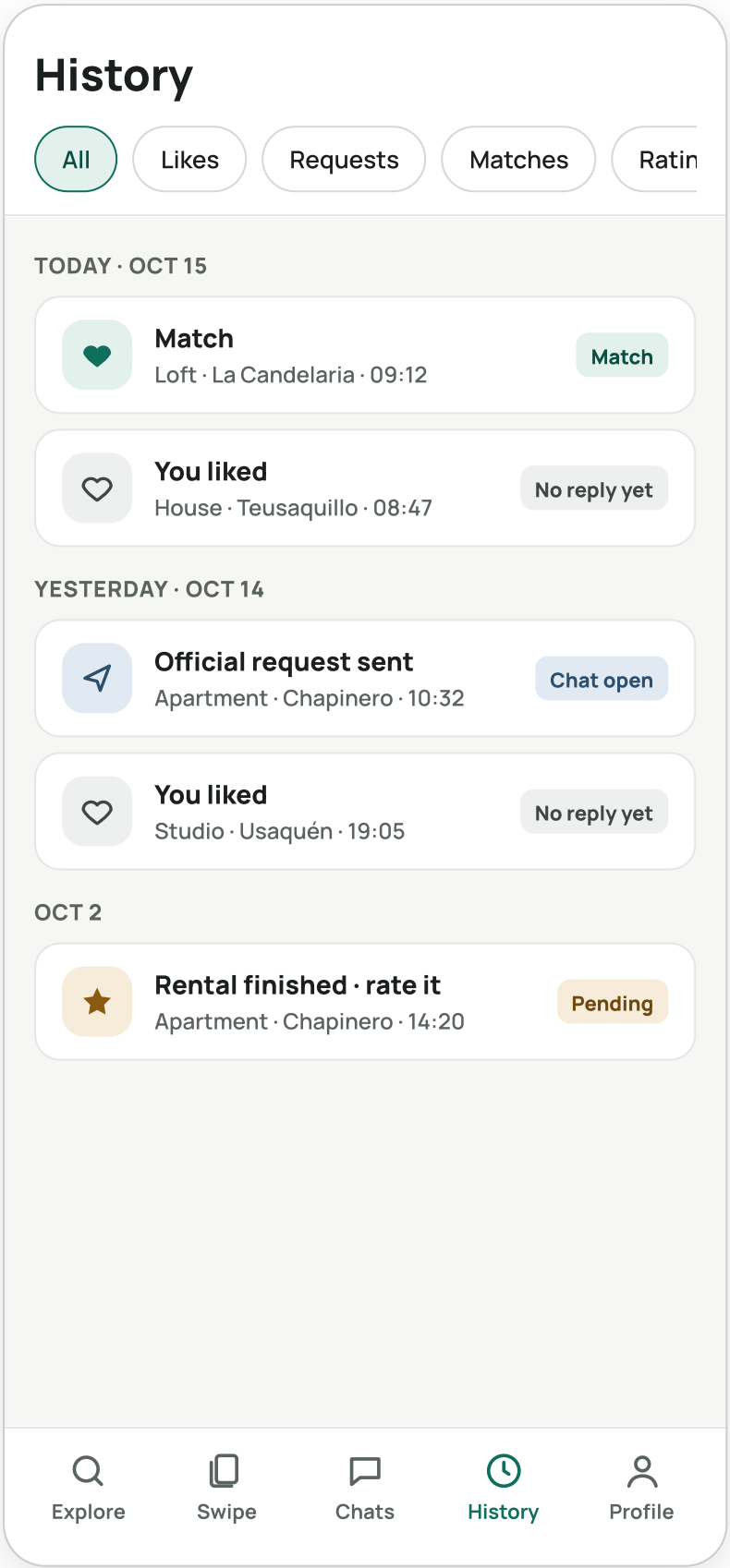
1–5 stars and an optional comment about the other party, shown on their public profile.



16 Interaction history

HU-14

Timeline of likes, requests, matches and ratings with date, time and result. Rejected likes show as “No reply yet” (RNF-12).



17 Admin panel

HU-15, HU-16, HU-17

Desktop back office for verification tickets, user reports (suspend or block) and the ranking-algorithm weights.

